

Introduction to Boolean Logic

From: <https://www.i-programmer.info/babbages-bag/235-logic-logic-everything-is-logic.html>

Logic, logic everywhere

Computers and logic are inseparable - right?

They are now but at the start things were much more hazy.

The first computers were conceived as automatic arithmetic engines and while their creators were aware that logic had something to do with it all, they weren't 100% clear as to the how or why.

Even today we tend to be over simplistic about logic and its role in computation and understanding the world and George Boole the man who started it all off was a bit over the top with the titles of his books on the subject -

Mathematical Analysis of Thought and An Investigation of the Laws of Thought.

Boole's work certainly started modern logic off on the right road, but it certainly wasn't anything to do with the "laws of thought". The fact of the matter is that even today we have no clear idea what laws govern thought and if we did the whole subject of artificial intelligence would be a closed one.

What George Boole did to be recognized as the father of modern information technology was to come up with an idea that was at the same time revolutionary and simple.

This video, a trailer for a documentary celebrating the bicentenary of his birth on November 2, 1815 hints at how his radical discovery underpins the digital age:

<https://youtu.be/aEjziLv-YjI>

Who was George Boole?

A contemporary of [Charles Babbage](#), whom he briefly met, Boole is these days credited as being the "forefather of the information age". An Englishman by birth, in 1849 he became the first professor of mathematics in Ireland's new Queen's College (now University College) Cork.

He died at the age of 49 in 1864 and his work might never have had an impact on computer science without [Claude Shannon](#), who 70 years later recognised the relevance for engineering of Boole's symbolic logic. As a result, Boole's thinking has become the practical foundation of digital circuit design and the theoretical grounding of the the digital age.

Boolean Logic

Boolean logic is very easy to explain and to understand.

- You start off with the idea that some statement P is either true or false, it can't be anything in between (this called the law of the excluded middle).

- Then you can form other statements, which are true or false, by combining these initial statements together using the fundamental operators And, Or and Not.

Exactly what a "fundamental" operator is forms an interesting question in its own right - something we will return to later when we ask how few logical operators do we actually need?

The way that all this works more or less fits in with the way that we used these terms in English.

For example, if P is true then Not(P) is false So, if "today is Monday" is true then "Not(today is Monday)" is false.

We often translate the logical expression into English as "today is Not Monday" and this makes it easier to see that it is false if today is indeed Monday.

Are you following?

Well this is the problem with this sort of discussion. It very quickly becomes convoluted and difficult to follow and this is part of the power of Boolean logic. You can write down arguments clearly in symbolic form.

Truth Tables

The rules for combining expressions are usually written down as tables listing all of the possible outcomes. These are called truth tables and for the three fundamental operators these are:

P	Q	P AND Q
F	F	F
F	T	F
T	F	F
T	T	T

P	Q	P OR Q
F	F	F
F	T	T
T	F	T
T	T	T

P	NOT P
F	T
T	F

Notice that while the Boolean And is the same as the English use of the term, the Boolean Or is a little different.

When you are asked would you like "coffee OR tea" you are not expected to say yes to both!

In the Boolean case however "Or" most certainly includes both. When P is true and Q is true the combined expression (P Or Q) is also true.

There is a Boolean operator that corresponds to the English use of the term "or" and it is called the "Exclusive or" written as EOR or XOR. Its truth table is:

P	Q	P XOR Q
F	F	F
F	T	T
T	F	T
T	T	F

and this one really would stop you having both the tea and the coffee at the same time (notice the last line is True XOR True = False).

Practical truth tables

All this seems very easy but what value has it?

It most certainly isn't a model for everyday reasoning except at the most trivial "coffee or tea" level.

We do use Boolean logic in our thinking, well politicians probably don't but that's another story, but only at the most trivially obvious level.

However, if you start to design machines that have to respond to the outside world in even a reasonably complex way then you quickly discover that Boolean logic is a great help.

For example, suppose you want to build a security system which only works at night and responds to a door being opened. If you have a light sensor you can treat this as giving off a signal that indicates the truth of the statement:

P = It is daytime.

Clearly Not(P) is true when it is night-time and we have our first practical use for Boolean logic!

What we really want is something that works out the truth of the statement:

R= Burglary in progress

from P and

Q = Window open

A little raw thought soon gives the solution that

R = Not (P) And Q

That is the truth of “Burglary in progress” is given by the following truth table:

P	Q	NOT(P)	NOT(P)AND Q
F	F	T	F
F	T	T	T
T	F	F	F
T	T	F	F

From this you should be able to see that the alarm only goes off when it is night-time and a window opens.

From Truth Tables to Electronic Circuits

Not only does this little demonstration illustrate why Boolean logic is useful in designing such systems, it also explains why electronics circuits to perform Boolean logic are commonplace. You can buy integrated circuits that perform And, Or and Not and many other combinations of these operations in a single easy to use package.

Where is the “truth” in this?

Clearly Boolean logic works with “true” and “false” and this is certainly how Boole himself thought about it, however you can work with the same system of operators and results no matter what you call the two states - up/down, on/off or zero/one.

In the electronics case it is more natural to think of high and low voltage as representing the natural states that we are working with. An explanation that involves Boolean logic often sounds more appropriate when expressed in terms that belong to the underlying subject matter but it is still the same Boolean logic.

What has all of this got to do with computers?

The answer is two-fold. The first connection is that computers contain electronic circuitry that behaves in much the same way as the burglar alarm. In general Boolean logic helps when you need to design a circuit that has to give an output only when certain combinations of inputs are present.

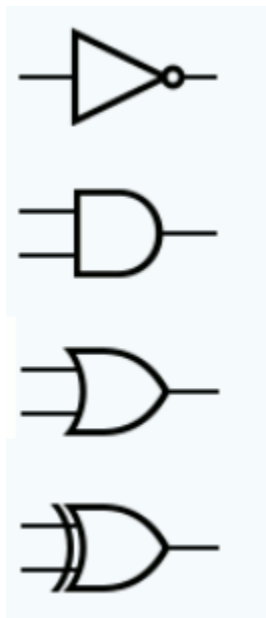
Such a circuit is called “combinatorial logic” and there are lots of them inside a computer. You can specify the behavior of any piece of combinatorial logic using a truth table. The only difference is that in most cases the tables are very large and converting them into efficient hardware is quite a difficult job.

Logic in Hardware

What makes Boolean logic so central to computers and a wide range of electronic devices is that logic is easy to implement. A logic "gate" is a piece of electronics that takes in some inputs and produces an output according to one of the logic tables. Usually the inputs are either high or low and this is interpreted as 1 and 0 or true and false.

For example a Not gate has a single input and its output is high when its input is low and vice versa. You can create more complicated logic circuits by wiring up gates.

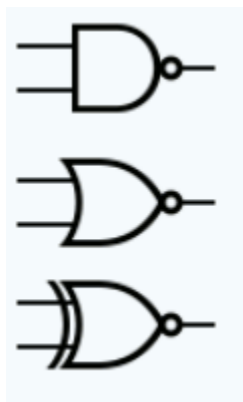
The standard symbols for the three basic logic gates - Not, And and Or plus the exclusive Or are:



Note that the European standard symbols are different but in the main these have not caught on.

In general a small circle on a connection indicates a negation or a not - see the Not gate.

For example, the symbols for NAND, NOR and Not EOR or XNOR are :

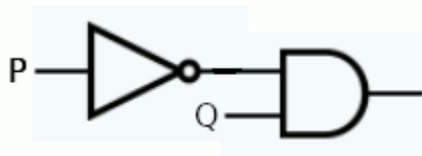


In many ways this is electronics made easy. You don't need to know anything about transistors or other components to build a logic circuit. Just work out what the logical expression is and wire gates up to implement it.

For example, our burglar alarm example has the logical expression:

$$R = \text{Not}(P) \text{ And } Q$$

So to implement it in hardware we need a Not gate and an AND gate:



Real logic circuits are much more complicated than this and can make use hundreds or even thousands of gates.

Binary arithmetic

The second connection is actually just a special case of the first but it is very important and it often confuses people. Boolean logic can be used to implement binary arithmetic.

Notice that there is no suggestion that binary arithmetic and Boolean logic are the same thing - they aren't. Binary arithmetic is just an example of a place value system for representing values. That is, we humans work in base 10 and computers find it easier to work in base two because they don't have ten fingers.

However when you do binary arithmetic you follow standard rules that determine how you should combine bits together to produce result bits. These rules can be expressed as combinatorial logic - in other words a truth table.

You can easily construct a table showing how to add two bits together to give a result and a carry to the next place:

A	B	Result	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

If you don't know how to add two bits together then simply accept that the table as your instructions for how to do it.

You can see that this is just another truth table and the combinatorial logic needed for it can be produced in the usual way. Actually this is only a [half adder](#) - yes this is the real technical term - in that it doesn't add a carry bit that might have been generated by a previous pair of bits.

The table for a "full adder", this too is the correct technical term is:

A	B	C	Result	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

So combinatorial circuits are the main connection between Boolean logic and computer hardware but there is more to computer hardware than combinatorial logic.

Occasionally you will hear computers referred to just one huge piece of Boolean logic but this is an overstatement. Large chunks are just And, Or and Not gates but there are also huge chunks that are not describable in terms of pure Boolean logic.

Flip-flops

Computers also include "sequential" logic, which includes an element of time.

For example, a flip-flop, again perfectly good jargon, is a circuit that changes state like a pendulum flips from side to side.

The odd thing is that you can make a flip-flop from two Not gates and it is true that most sequential logic can be understood in terms of combinations of logic gates. However, this is an incomplete explanation and requires something beyond Boolean logic.

For example, Boolean logic cannot cope with the statement:

$$A = \text{NOT } A$$

If A is true then NOT A is false and so A must be false but this means that NOT A is true, which means that A must be true, which means, and so on...

So from the point of view of Boolean logic this is nonsense.

Now imagine that the on one side of a blackboard you write "A is true" and on the other side you write "Not A is true". Now when you read "A is true" you are happy with this. When you turn the blackboard over you see "Not A is true" and so conclude A is false. As you keep flipping the board over the value of A flips from true to false and back again.

This turning over of the blackboard is the extra part of the theory needed to deal with sequential logic - it shows how time changes things. So computer hardware is not just one large statement in Boolean logic, which just happens to be true or false.

Its state changes with every tick of its system clock.

Boolean logic in programming

Boolean logic is fundamental to the design of computer hardware even if it isn't the whole story. The same holds true for programming.

A program also needs the element of time built into it to make it work and this takes it beyond the bounds of simple Boolean logic. However there are times when simple common sense reasoning lets even the best programmer down.

For example, a common task is making a decision using an IF statement:

```
IF (A>0 AND A<10) THEN do something
```

The bracket following the IF is almost pure Boolean logic and the something only gets done if it works out to be true.

So far so simple, so simple in fact that many programmers decide that they don't need to know anything about Boolean logic at all; a serious error.

Consider how you would change this example so that the "something" was done when the condition was false. The simplest way is to write:

```
IF NOT(A>0 AND A<10) THEN do something
```

However this offends some programmers who want to rewrite the condition not using the NOT.

Try it and see what you come up with.

A common mistake is to use:

```
IF (A<=0 AND A>=10) THEN do something
```

This isn't correct and if you don't believe it simply write out the truth table for both. The correct NOT of the condition is:

```
IF (A<=0 OR A>10) THEN do something
```

The switch from AND to OR shocks many experienced programmers and it is a source of many programming errors.

My advice is that if you can write a logical condition in a natural form but you really need the NOT of it then just write NOT in front of it!

Otherwise learn De Morgan's laws:

$\text{NOT}(A \text{ AND } B) = \text{NOT}(A) \text{ OR } \text{NOT}(B)$

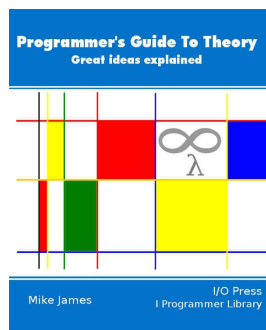
and

$\text{NOT}(A \text{ OR } B) = \text{NOT}(A) \text{ AND } \text{NOT}(B)$

Boolean logic is important as a fundamental way of thinking about very simple situations which involve combinations of two states.

What it is not is a theory of thought or of the way computers work. It is not even essential to designing or using a computer but it does make both tasks easier.

If you're serious about computing you need to be at home with Boolean logic in much the same way that you are at home with arithmetic - [binary arithmetic](#) of course.



The Universal Gate

One final and slightly deep thought.

How many operators do you need to implement Boolean logic?

The obvious answer is three - AND, OR and NOT - but this isn't correct.

The correct answer is that you only need one operation. Clearly it can't be just any operation. For example, it is impossible to make an OR from combinations of AND - try it.

However if you pick an operation that includes an element of the NOT operator then you can make anything you like using it.

For example the AND operator isn't universal but the NAND, i.e. Not AND is. The truth table for NAND is just $\text{NOT}(P \text{ AND } Q)$:

P	Q	P NAND Q
F	F	T

F	T	T
T	F	T
T	T	F

To make a NOT all you have to do is write:

$$P \text{ NAND } P = \text{NOT}(P)$$

P	P	P NAND P
F	F	T
T	T	F

Once you have a NOT you can make an AND:

$$\text{NOT}(P \text{ NAND } Q) = P \text{ AND } Q$$

The only operation that remains is to make an OR out of NAND and NOT and the solution to this is one of De Morgan's laws:

$$\text{NOT}(P \text{ AND } Q) = \text{NOT}(P) \text{ OR } \text{NOT}(Q)$$

A little work soon gives:

$$\text{NOT}(P) \text{ NAND } \text{NOT}(Q) = P \text{ OR } Q$$

So with a NAND operator you can build a NOT, an AND and an OR. This means that a single operator does everything that the usual three can do.

In practical terms this means that every circuit in a computer could be built using just a NAND gate. In practice electronic engineers like to use a variety of gates because it is simpler.

NAND isn't the only universal operator.

You might guess that NOR, i.e. $\text{NOT}(P \text{ OR } Q)$ is also universal but what about XOR? There isn't a hint of a NOT gate here - is there?

However, what about $P \text{ XOR } \text{True}$?

P	True	P XOR P
F	T	T
T	T	F

Yes, $P \text{ XOR True} = \text{NOT } P$ and from this you can build an AND and an OR operator. XOR is also a universal operator for Boolean logic.

It is something to think about that the whole of logic can be built from just one operation.